

# Sovereign Health Data Pipeline Engine

ENTERPRISE OPENEHR-TO-CDM BLUEPRINT, STRATEGY NARRATIVE,  
& OPERATIONAL RUNBOOK

|                    |                                                                                                                              |
|--------------------|------------------------------------------------------------------------------------------------------------------------------|
| Document Ref:      | HS-EHR-CDM-COMPREHENSIVE-SPEC                                                                                                |
| Version:           | 1.0.0-Production Release                                                                                                     |
| Principal Advisor: | Muhammad Arshad                                                                                                              |
| Contact Email:     | pmoarshaduk@gmail.com                                                                                                        |
| Target Audience:   | Chief Information Officers (CIOs), Chief Data Officers (CDOs), Enterprise Architects, Hospital Digital Transformation Boards |

# 1. Executive Narrative & Strategy Framework

---

When architecting enterprise health data networks across European landscapes—whether navigating regional digital modernizations or specific healthcare development mandates—vendor lock-in and brittle relational database schemas remain primary drivers of high maintenance costs and technical debt. Relational database architectures (SQL) introduce systemic inefficiencies because clinical guidelines and healthcare analytics parameters change faster than enterprise schemas can safely be refactored.

This document establishes a sustainable operational blueprint by completely decoupling clinical knowledge modeling from the persistence layer. By deploying an openEHR-compliant Clinical Data Repository (CDR) as the core data persistence vault, frontline forms are validated directly at the API gateway through Operational Templates (OPTs). This ensures total separation of concerns: healthcare applications can evolve, add parameters, or deprecate nodes (shifting from Version 1 to Version 2) without requiring single table modifications, database refactoring, or down-time deployments.

To support high-performance secondary data execution, analytics dashboards, and regional data spaces, this specification outlines an asynchronous, semantic-aware Extract-Transform-Load (ETL) infrastructure engine. Leveraging Archetype Query Language (AQL), the pipeline extracts deeply nested tree path coordinates and maps them immediately to flat relational Clinical Data Model (CDM) rowsets benchmarked against global standards (\*HS\* parameters).

# 2. End-to-End Patient Data Lifecycle Matrix

---

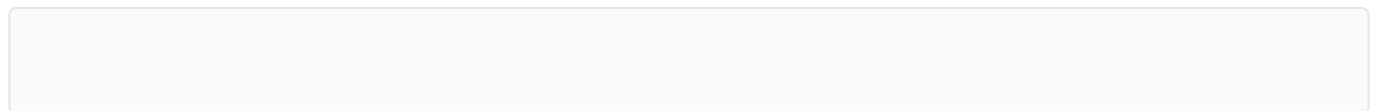
The matrix below maps a critical data stream—an acute myocardial infarction (STEMI) pre-hospital triage event—across the five key operational stages of the enterprise healthcare topology:

| Stage                 | System / Node                     | Data Format             | Processing Environment & Actions                                                                                                                                             | Governance / Baseline                                  |
|-----------------------|-----------------------------------|-------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| <b>1. Capture</b>     | Pre-Hospital Mobile Triage Client | Native UI Form Object   | Paramedics input initial clinical metrics, emergency parameters, and a comprehensive suite of NEWS2 vital observations.                                                      | openEHR Archetype Bound (`OBSERVATION.news2.v1`)       |
| <b>2. Ingestion</b>   | openEHR CDR Gateway Layer         | Structured JSON Payload | Data is transmitted securely over an encrypted HTTPS POST link. The gateway performs automatic programmatic schema checks against the operational template.                  | HTTPS TLS 1.3, openEHR REST API Standard               |
| <b>3. Persistence</b> | Core Core Repository Engine       | Binary JSON (`JSONB`)   | The validated document is safely written into the patient's private `EHR_ID` ledger. The file is issued an immutable `Composition_UID` and version stamp (`::1`).            | ACID Transactional Security, ISO 21090 Model Integrity |
| <b>4. Integration</b> | Semantic ETL Middleware Runtime   | In-Memory Dictionary    | The ETL service triggers on an incremental time-window checkpoint. It runs an AQL script to isolate updates, extracts nested data trees, and trims whitespace text elements. | Data Provenance and Lineage Audit, Safe Type Casting   |
| <b>5. Analytics</b>   | Clinical Data Warehouse (CDW)     | Flat Relational Row     | The normalized, flat rowset is written via an idempotent upsert straight into the destination analytics layout, tracking histories flawlessly.                               | *HS* Schema Dict Benchmark, GDPR Compliant             |

## 3. Technical Architecture Diagrams

### 3.1 Conceptual Structural Topology

The conceptual model handles validation directly at the input gateway layer, isolating back-end physical storage rows from applications.



### 3.2 Logical Data Flow Matrix

The asynchronous engine leverages AQL scripts to transform hierarchical trees into relational columns.

## 4. Production Script Implementation

---

The code framework below represents the automated implementation runner. It queries the target platform backend, runs string formatting mappings, and structures clinical tree nodes into a clear target table file asset:

```

param (
    [string]$CdrUrl = "http://localhost:8080/ehrbase/rest/openehr/v1",
    [string]$OutputFilePath = "C:\projects\nhs_cdm_ambulance_export.csv"
)

$ErrorActionPreference = "Stop"
$StartTime = Get-Date

# Phase 1: Semantic Extraction Layer via AQL
$AqlPayload = @{
    q = "SELECT
        e/ehr_id/value AS ehr_id, c/uid/value AS composition_id, c/context/start_time/
value AS encounter_time,
        diag/data[at0001]/items[at0002]/value/value AS diagnosis_name,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0021]/value AS
respiration_rate,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0033]/value AS
spo2_scale1_code,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0061]/value AS
spo2_scale2_code,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0037]/value AS
air_or_oxygen,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0017]/value AS
systolic_bp,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0010]/value AS
heart_pulse_rate,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0025]/value AS
level_of_consciousness,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0066]/value AS
body_temperature,
        news/data[at0001]/events[at0002]/data[at0003]/items[at0060]/value/
defining_code/code_string AS risk_category_code
    FROM EHR e CONTAINS COMPOSITION c CONTAINS (
        EVALUATION diag[openEHR-EHR-EVALUATION.problem_diagnosis.v1] OR
        OBSERVATION news[openEHR-EHR-OBSERVATION.news2.v1]
    )"
} | ConvertTo-Json

$Headers = @{ "Accept" = "application/json"; "Content-Type" = "application/json" }

try {
    $ApiResponse = Invoke-WebRequest -Uri "$CdrUrl/query/aql" -Method POST -Headers
$Headers -Body $AqlPayload -UseBasicParsing
    $DataRows = (ConvertFrom-Json $ApiResponse.Content).rows
    $ProcessedPayloadBatch = @()

    foreach ($CurrentRow in $DataRows) {
        $CdmNormalizedRecord = [ordered]@{
            Target_EHR_ID          = $CurrentRow[0]
            Source_Document_UID   = $CurrentRow[1]
            Encounter_Timestamp    = if ($CurrentRow[2] -ne $null)
{ [DateTime]::Parse($CurrentRow[2]).ToString("yyyy-MM-dd HH:mm:ss") } else { $null }
            Condition_Title       = if ($CurrentRow[3] -ne $null)
{ $CurrentRow[3].ToUpper().Trim() } else { "NOT SPECIFIED" }
            Respiration_Rate_BPM  = if ($CurrentRow[4] -ne $null)
{ $CurrentRow[4] } else { $null }
            SpO2_Scale_1_Status   = if ($CurrentRow[5] -ne $null)
{ $CurrentRow[5] } else { $null }
            SpO2_Scale_2_Status   = if ($CurrentRow[6] -ne $null)

```

```

{ $CurrentRow[6] } else { $null }
    Air_Or_Oxygen_Administered = if ($CurrentRow[7] -ne $null)
{ $CurrentRow[7] } else { $null }
    Systolic_Blood_Pressure_mmHg = if ($CurrentRow[8] -ne $null)
{ $CurrentRow[8] } else { $null }
    Heart_Pulse_Rate_BPM = if ($CurrentRow[9] -ne $null)
{ $CurrentRow[9] } else { $null }
    Level_Of_Consciousness = if ($CurrentRow[10] -ne $null)
{ $CurrentRow[10] } else { $null }
    Body_Temperature_Celsius = if ($CurrentRow[11] -ne $null)
{ $CurrentRow[11] } else { $null }
    NEWS2_Risk_Category_Code = if ($CurrentRow[12] -ne $null)
{ $CurrentRow[12] } else { $null }
    Pipeline_Executed_At = (Get-Date -Format "yyyy-MM-dd HH:mm:ss")
    Record_Status = "ACTIVE"
}
$ProcessedPayloadBatch += [PSCustomObject]$CdmNormalizedRecord
}
$ProcessedPayloadBatch | Export-Csv -Path $OutputFilePath -NoTypeInfoation -Encoding
UTF8
Write-Host "● HEALTH DATA ETL PIPELINE EXECUTED SUCCESSFULLY" -ForegroundColor Green
} catch {
    Write-Host "✗ INFRASTRUCTURE PIPELINE CATASTROPHIC FAILURE" -ForegroundColor Red
    exit 1
}

```

## 5. Verification Runbook & Operational Procedures

To safely validate the runtime components and execute infrastructure checks, follow these standard verification rules:

### Step 1: Active Endpoint Connection Verification

Execute the core engine file while the backend repository link is active. Verify the console displays a successful connection and extracts rows without dropping API requests.

### Step 2: Schema Field Structure Audit

Open the physical written dataset file (`nhs\_cdm\_ambulance\_export.csv`) inside a tabular validation editor or relational engine. Verify that all components of the clinical telemetry—specifically tracking nodes like **Heart\_Pulse\_Rate\_BPM**—have mapped cleanly to explicit relational schema header elements.

### Step 3: Defensive Structural Null Evaluation

Push an active edge-case document into the repository gateway that deliberately omits optional vital sign observations (e.g., missing respiratory rate attributes). Run the pipeline engine. Verify that the transformation service processes the missing parameters safely without throwing runtime schema faults, outputting a clear, structured empty indicator into the destination analytics cell.

## 6. Architecture Standards & Material References

---

- **openEHR Reference Model Specification (v1.0.2):** Structural definition frameworks, clinical tree mapping rules, and Archetype Query Language (AQL) pathways. Portal access via: [specifications.openehr.org](https://specifications.openehr.org).
- **Sovereign Repository Design Standards:** Deep execution rules and architectural optimizations for handling semi-structured binary text blocks inside relational engines. Documentation available at: [ehrbase.org](https://ehrbase.org).
- **SNOMED International Core Systems:** Unified global medical ontologies and semantic concept validation mapping sets. Implementation matrices located at: [snomed.org](https://snomed.org).